

Hardware Implementation of Neural Network Inference Using SystemVerilog

This project consists of **four** phases. Phases are explained as follows:

Phase 1: Basics and Implementation of a Simple MAC

In the first phase, the goal is to familiarize yourself with SystemVerilog and basic RTL concepts and to create a simple computational block.

What to do:

Design a simple MAC unit. MAC stands for:

Multiply-accumulate

A circuit that multiplies two numbers and adds them to a previous value.

Its general form is:

$$\text{acc} = \text{acc} + a * b$$

In this phase, you will need to:

- * Write a MAC module in SystemVerilog
- * Support signed numbers
- * Use clocks and resets
- * Write a testbench to test the MAC
- * Take a waveform and explain what happens at each clock
- * Implement a simple ReLU

ReLU is very simple:

If the number is negative, the output is zero.

If it is positive, the same output is given.

At the end of this phase, it should explain what MAC is, why it is important in a neural network, what the inputs are, the output, and how many clocks the result is produced.

Phase 2: Build a simple model in Python and convert weights to Fixed-Point

In the second phase, you build or train a very small neural network in Python. The purpose of Python is only to have a reference model and get the weights. The main part of the project is still SystemVerilog.

The proposed network should be very small, for example:

2 inputs

4 neurons in the hidden layer

2 outputs

That is, the overall structure of the network:

$2 \rightarrow 4 \rightarrow 2$

You can build a simple 2D dataset. For example, a few points with x and y coordinates that belong to two different classes. It is not necessary to use a heavy or famous dataset like MNIST. It is even better to keep the dataset simple and small to focus on the hardware.

In this phase you should do the following:

- * Create a simple dataset in Python
- * Train a small neural network
- * Extract weights and biases
- * Save Python outputs for multiple test inputs
- * Convert weights and inputs from decimal to fixed-point

In this project we will not use floating-point, because it is complicated to implement in hardware to begin with. Instead we will use fixed-point. That is, we will convert decimal numbers to integers with a specific scale.

For example, if scale = 16:

1.5 will be converted to 24

-0.75 will be converted to -12

The goal of this phase is to understand what the network outputs in software and then recreate the same output to an acceptable level in SystemVerilog.

Phase 3: Implementing a neuron and then a layer in SystemVerilog

In the third phase, you will enter the main hardware implementation.

First, you need to design a simple neuron in SystemVerilog. A neuron does the following:

Takes inputs

Multiplies each input by its own weight

Sums the results

Adds bias

Applies an activation function such as ReLU

Produces an output

General form:

$$y = \text{ReLU}(w_1 * x_1 + w_2 * x_2 + b)$$

Or if the number of inputs increases:

$$y = \text{ReLU}(w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + \dots + b)$$

In this phase, it is better to design modularly. That is, your code is not just one big file. For example, you have these modules:

- * mac_unit.sv
- * relu.sv
- * neuron.sv
- * dense_layer.sv
- * testbench

After implementing a neuron, you need to create a small layer. For example, a layer with 4 neurons. Each neuron can produce its own output.

In this phase, you need to write a testbench and compare the output of a neuron or a layer with the Python output. The outputs may not be exactly equal, because in Python we usually have floats, but in SystemVerilog we have fixed-point. This difference is normal, but you should be able to explain it.

Phase 4: Building the complete network, final testing and reporting

In the fourth phase, you put the previous parts together and build a complete small neural network.

The proposed structure is:

$$2 \rightarrow 4 \rightarrow 2$$

That is:

2 inputs enter the network

The hidden layer is calculated with 4 neurons

ReLU is applied

The output layer is calculated with 2 neurons

Finally, it is decided whether the output belongs to class 0 or class 1

For the final decision, you can use simple argmax. That is, if the first output was larger, class 0 and if the second output was larger, class 1.

In this phase, you should deliver the following:

- * Complete SystemVerilog code
- * Complete testbench
- * Several test inputs, at least 10 examples
- * Comparison of Python and SystemVerilog output
- * Waveform from circuit execution
- * Explanation of the number of clocks it takes to get the output ready
- * Short report on the project architecture
- * Explanation of how you chose fixed-point
- * Explanation of the limitations of the project and what can be done to improve it

If you want to make the project a little stronger, you can check one of the following in the optional section:

- * Comparison of serial and parallel architectures
- * Investigate the effect of the number of fixed-point bits on the accuracy of the output
- * Add a simple pipeline
- * More automated testing
- * Implementation on FPGA, of course, this is not required